

EE-2003 Computer Organization and Assembly Language (COAL)

Week 2
Dr. Shakil Ahmed

Contents

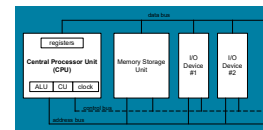
- General Concepts
- IA-32 Processor Architecture
- IA-32 Memory Management
- 64-bit Processors
- Components of an IA-32 Microcomputer
- Input-Output System

General Concepts

- Basic microcomputer design
- Instruction execution cycle
- Reading from memory
- How programs run

Basic Microcomputer Design

- clock synchronizes CPU operations
- control unit (CU) coordinates sequence of execution steps
- ALU performs arithmetic and bitwise processing

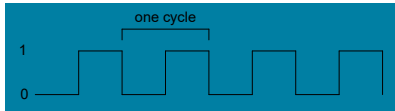


- The CPU is attached to the rest of the computer via pins attached to the CPU socket in the computer's motherboard. Most pins connect to the data bus, the control bus, and the address bus.
- The memory storage unit is where instructions and data are held while a computer program is running. The storage unit receives requests for data from the CPU, transfers data from random access memory (RAM) to the CPU, and transfers data from the CPU into memory.
- All processing of data takes place within the CPU, so programs residing in memory must be copied into the CPU before they can execute. Individual program instructions can be copied into the CPU one at a time, or groups of instructions can be copied together

- A bus is a group of parallel wires that transfer data from one part of the computer to another.
- A computer system usually contains four bus types: data, I/O, control, and address.
- The data bus transfers instructions and data between the CPU and memory.
- The I/O bus transfers data between the CPU and the system input/output devices.
- The control bus uses binary signals to synchronize actions of all devices attached to the system bus.
- The address bus holds the addresses of instructions and data when the currently executing instruction transfers data between the CPU and memory

Clock

- synchronizes all CPU and BUS operations
- machine (clock) cycle measures time of a single operation
- clock is used to trigger events



- The duration of a clock cycle is calculated as the reciprocal of the clock's speed, which in turn is measured in oscillations per second. A clock that oscillates 1 billion times per second (1 GHz), for example, produces a clock cycle with a duration of one billionth of a second (1 nanosecond).
- A machine instruction requires at least one clock cycle to execute, and a few require in excess of 50 clocks (the multiply instruction on the 8088 processor, for example).
- Instructions requiring memory access often have empty clock cycles called wait states because of the differences in the speeds of the CPU, the system bus, and memory circuits

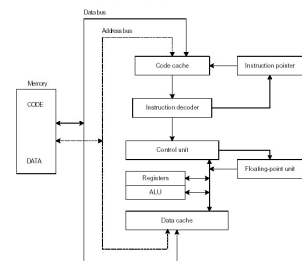
Instruction Execution Cycle

- A single machine instruction does not just magically execute all at once. The CPU has to go through a predefined sequence of steps to execute a machine instruction, called the instruction execution cycle.
- Let's assume that the instruction pointer register holds the address of the instruction we want to execute. Here are the steps to execute it

Instruction Execution Cycle

- Fetch
- Decode
- Fetch operands
- Execute
- Store output

Figure 2-2 Simplified Pentium CPU Block Diagram.



- The diagram helps to show relationships between components that interact during the instruction execution cycle.
- In order to read program instructions from memory, an address is placed on the address bus. Next, the memory controller places the requested code on the data bus, making the code available inside the code cache. The instruction pointer's value determines which instruction will be executed next.
- The instruction is analyzed by the instruction decoder, causing the appropriate digital signals to be sent to the control unit, which coordinates the ALU and floating-point unit.
- Although the control bus is not shown in this figure, it carries signals that use the system clock to coordinate the transfer of data between the different CPU components.

Reading from Memory

Multiple machine cycles are required when reading from memory, because it responds much more slowly than the CPU. The steps are:

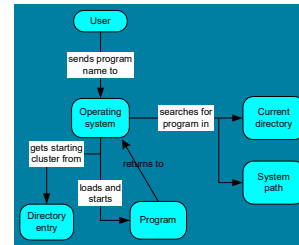
1. Place the address of the value you want to read on the address bus.
2. Assert (changing the value of) the processor's RD (read) pin.
3. Wait one clock cycle for the memory chips to respond.
4. Copy the data from the data bus into the destination operand

Each of these steps generally requires a single clock cycle, a measurement of time based on a clock that ticks inside the processor at a regular rate.

Cache Memory

- High-speed expensive static RAM both inside and outside the CPU.
 - Level-1 cache: inside the CPU
 - Level-2 cache: outside the CPU
- Cache hit: when data to be read is already in cache memory
- Cache miss: when data to be read is not in cache memory.

How a Program Runs



IA-32 Processor Architecture

- Modes of operation
- Basic execution environment
- Floating-point unit

Modes of Operation

These modes define how the processor operates, manages memory, and controls system resources.

- **Real-address mode**
 - native MS-DOS (original operating mode)
 - This mode is useful if a program requires direct access to system memory and hardware devices.
 - Only one program runs at a time
 - Used mainly for bootstrapping (BIOS starts in Real Mode).
 - **Protected mode**
 - native mode (Windows, Linux) (introduced with Intel 80286)
 - In this mode all instructions and features are available. Programs are given separate memory areas named segments, and the processor prevents programs from referencing memory outside their assigned segments
 - Provides virtual memory, and multitasking.
 - Uses segmentation and paging for memory management.
 - Allows multiple programs to run simultaneously (multitasking).
 - Modern operating systems (Windows, Linux) rely on protected mode.
 - **System management mode (SMM)**
 - power management, system security, diagnostics
 - These functions are usually implemented by computer manufacturers who customize the processor for a particular system setup.
- **Virtual-8086 mode** (special case of protected mode, introduced with 80386) While in protected mode, the processor can directly execute real-address mode software such as MS-DOS programs in a safe environment. In other words, if a program crashes or attempts to write data into the system memory area, it will not affect other programs running at the same time. A modern operating system can execute multiple separate virtual-8086 sessions at the same time.

• Long Mode (x86-64 processors)

- Used in 64-bit processors.
- **Characteristics:**
 - Supports 64-bit addressing (up to 16 EB theoretically, ~256 TB practically).
 - Includes both **64-bit mode** (for modern OS/apps) and **compatibility mode** (for 32-bit apps).
- **Other Modes**
 - **Unreal Mode:** A hack combining Real Mode with extended segment limits.
 - **Compatibility Mode:** Part of long mode for running 16-bit/32-bit code

Basic Execution Environment

- Addressable memory
- General-purpose registers
- Index and base registers
- Specialized register uses
- Status flags
- Floating-point, MMX, XMM registers

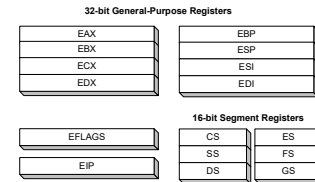
Addressable Memory (Address Space)

- Protected mode (32-bit)
 - 4 GB
 - 32-bit address
- Real-address and Virtual-8086 modes
 - Maximum 1 MB memory addressable
 - 20-bit address bus

General-Purpose Registers

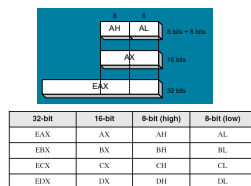
Named storage locations inside the CPU, optimized for speed.

There are eight general-purpose registers, six segment registers, a processor status flags register (EFLAGS), and an instruction pointer (EIP).



Accessing Parts of Registers

- Use 8-bit name, 16-bit name, or 32-bit name
- Applies to EAX, EBX, ECX, and EDX



Index and Base Registers

- Some registers have only a 16-bit name for their lower half:

32-bit	16-bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP

Some Specialized Register Uses (1 of 2)

- General-Purpose**
 - EAX – accumulator
 - EAX is automatically used by multiplication and division instructions. It is often called the extended accumulator register.
 - EBX (Base Register)
 - Often used to store base addresses for memory access.
 - ECX – loop counter or shift/rotate instructions
 - EDX (Data Register)
 - Used in multiplication, division, and I/O operations.
 - ESP – stack pointer
 - It is rarely used for ordinary arithmetic or data transfer. It is often called the extended stack pointer register.
 - ESI, EDI – index registers
 - used by high-speed memory transfer instructions. They are sometimes called the extended source index and extended destination index registers.
 - EBP – extended frame pointer (stack)
 - used by high-level languages to reference function parameters and local variables on the stack. It should not be used for ordinary arithmetic or data transfer except at an advanced level of programming. It is often called the extended frame pointer register.

- In real-address mode, 16-bit segment registers indicate base addresses of preassigned memory areas named segments. In protected mode, segment registers hold pointers to segment descriptor tables. Some segments hold program instructions (code), others hold variables (data), and another segment named the stack segment holds local function variables and function parameters.

- Segment Registers**
 - CS – code segment
 - DS – data segment
 - SS – stack segment
 - ES, FS, GS – additional segments

Some Specialized Register Uses (2 of 2)

- **EIP – instruction pointer**
 - The EIP, or instruction pointer, register contains the address of the next instruction to be executed. Certain machine instructions manipulate EIP, causing the program to branch to a new location.
- **EFLAGS**
 - The EFLAGS (or just Flags) register consists of individual binary bits that control the operation of the CPU or reflect the outcome of some CPU operation. Some instructions test and manipulate individual processor flags
 - each flag is a single binary bit

- **Control Flags**
 - Control flags control the CPU's operation. For example, they can cause the CPU to break after every instruction executes, interrupt when arithmetic overflow is detected, enter virtual-8086 mode, and enter protected mode.
 - Programs can set individual bits in the EFLAGS register to control the CPU's operation.
 - Examples are the Direction and Interrupt flags.

- **Difference Between GPRs and Special Purpose Registers**
- **GPRs:** Can store any data and are not limited to a specific function.
- **Special Purpose Registers:** Have predefined roles, e.g.:
 - **IP/RIP** – Instruction Pointer
 - **FLAGS/RFLAGS** – Status flags
 - **CS, DS, SS, ES, FS, GS** – Segment registers
 - **Control Registers (CR0–CR4)** – Memory management, protection, etc.

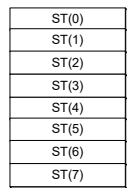
Status Flags

The status flags reflect the outcomes of arithmetic and logical operations performed by the CPU.

- **Carry**
 - unsigned arithmetic out of range
- **Overflow**
 - signed arithmetic out of range
- **Sign**
 - result is negative
- **Zero**
 - result is zero
- **Auxiliary Carry**
 - carry from bit 3 to bit 4 in an 8-bit operand.
- **Parity**
 - It is set if the least-significant byte in the result contains an even number of 1 bits. Otherwise, PF is clear. In general, it is used for error checking when there is a possibility that data might be altered or corrupted

Floating-Point, MMX, XMM Registers

- **Eight 80-bit floating-point data registers**
 - ST(0), ST(1), . . . , ST(7)
 - arranged in a stack
 - used for all floating-point arithmetic
- **Eight 64-bit MMX registers**
- **Eight 128-bit XMM registers for single-instruction multiple-data (SIMD) operations**



What's Next (2 of 5)

- General Concepts
- IA-32 Processor Architecture
- **IA-32 Memory Management**
- 64-Bit Processors
- Components of an IA-32 Microcomputer
- Input-Output System

IA-32 Memory Management

- Real-address mode
- Calculating linear addresses
- Protected mode
- Multi-segment model
- Paging

Protected Mode

- 4 GB addressable RAM
 - (00000000 to FFFFFFFFh)
- Each program assigned a memory partition which is protected from other programs
- Designed for multitasking
- Supported by Linux & MS-Windows

64-Bit Processors

- 64-Bit Operation Modes
 - Compatibility mode – can run existing 16-bit and 32-bit applications (Windows supports only 32-bit apps in this mode)
 - 64-bit mode – Windows 64 uses this
- Basic Execution Environment
 - addresses can be 64 bits (48 bits, in practice)
 - 16 64-bit general purpose registers
 - 64-bit instruction pointer named RIP

64-Bit General Purpose Registers

- 32-bit general purpose registers:
 - EAX, EBX, ECX, EDX, EDI, ESI, EBP, ESP, R8D, R9D, R10D, R11D, R12D, R13D, R14D, R15D
- 64-bit general purpose registers:
 - RAX, RBX, RCX, RDX, RDI, RSI, RBP, RSP, R8, R9, R10, R11, R12, R13, R14, R15

Big-Endian and Little-Endian

- **Endianness** refers to the order in which a sequence of bytes (e.g., from a multi-byte encoded number) is stored or transmitted. The difference lies in which "end" of the number comes first:

Order Type	Description	Example (0x12345678)
Big-Endian	Most significant byte (MSB) stored at the lowest memory address—i.e., "big end" comes first	Memory: 12 34 56 78
Little-Endian	Least significant byte (LSB) stored at the lowest memory address—i.e., "little end" first.	Memory: 78 56 34 12

Intel Processors follow Little-Endian Format

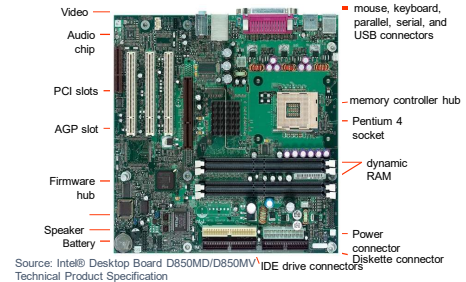
Components of an IA-32 Microcomputer (Self Study)

- Motherboard
- Video output
- Memory
- Input-output ports

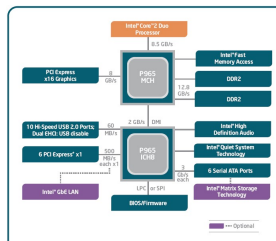
Motherboard

- CPU socket
- External cache memory slots
- Main memory slots
- BIOS chips
- Sound synthesizer chip (optional)
- Video controller chip (optional)
- IDE, parallel, serial, USB, video, keyboard, joystick, network, and mouse connectors
- PCI bus connectors (expansion cards)

Intel D850MD Motherboard



Intel 965 Express Chipset



Video Output

- Video controller
 - on motherboard, or on expansion card
 - AGP ([accelerated graphics port technology](#))*
- Video memory (VRAM)
- Video CRT Display
 - uses raster scanning
 - horizontal retrace
 - vertical retrace
- Direct digital LCD monitors
 - no raster scanning required

* This link may change over time.

Sample Video Controller (ATI Corp.) (1 of 2)

- 128-bit 3D graphics performance powered by RAGE™ 128 PRO
- 3D graphics performance
- Intelligent TV-Tuner with Digital VCR
- TV-ON-DEMAND™
- Interactive Program Guide
- Still image and MPEG-2 motion video capture
- Video editing
- Hardware DVD video playback
- Video output to TV or VCR

Sample Video Controller (ATI Corp.) (2 of 2)



Memory (1 of 2)

- ROM
 - read-only memory
- EPROM
 - erasable programmable read-only memory
- Dynamic RAM (DRAM)
 - inexpensive; must be refreshed constantly
- Static RAM (SRAM)
 - expensive; used for cache memory; no refresh required

Memory (2 of 2)

- Video RAM (VRAM)
 - dual ported; optimized for constant video refresh
- CMOS RAM
 - complimentary metal-oxide semiconductor
 - system setup information
- See: [Intel platform memory](#) (Intel technology brief: link address may change)

Input-Output Ports

- USB (universal serial bus)
 - intelligent high-speed connection to devices
 - up to 12 megabits/second
 - USB hub connects multiple devices
 - *enumeration*: computer queries devices
 - supports *hot* connections
- Parallel
 - short cable, high speed
 - common for printers
 - bidirectional, parallel data transfer
 - Intel 8255 controller chip

Input-Output Ports (cont)

- Serial
 - RS-232 serial port
 - one bit at a time
 - uses long cables and modems
 - 16550 UART (universal asynchronous receiver transmitter)
 - programmable in assembly language

Device Interfaces

- ATA host adapters
 - intelligent drive electronics (hard drive, CDROM)
- SATA (Serial ATA)
 - inexpensive, fast, bidirectional
- FireWire
 - high speed (800 MB/sec), many devices at once
- Bluetooth
 - small amounts of data, short distances, low power usage
- Wi-Fi (wireless Ethernet)
 - IEEE 802.11 standard, faster than Bluetooth

What's Next (5 of 5)

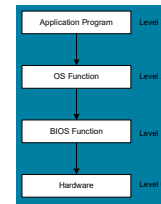
- General Concepts
- IA-32 Processor Architecture
- IA-32 Memory Management
- Components of an IA-32 Microcomputer
- **Input-Output System**

Levels of Input-Output

- Level 3: High-level language function
 - examples: C++, Java
 - portable, convenient, not always the fastest
- Level 2: Operating system
 - Application Programming Interface (API)
 - extended capabilities, lots of details to master
- Level 1: BIOS
 - drivers that communicate directly with devices
 - OS security may prevent application-level code from working at this level

Displaying a String of Characters

When a HLL program displays a string of characters, the following steps take place:



Programming levels

Assembly language programs can perform input-output at each of the following levels:

